

M2 MIC - Codes correcteurs - TD/TP 4

Exercice 1 : Sage Math

1. Implémenter une fonction `canal` qui simule un canal q -aire produisant exactement t erreurs.
2. Implémenter le codeur RS .
3. Implémenter le décodage de Berlekamp-Welch et de Berlekamp-Massey (voir TD/TP 2)
4. Comparer expérimentalement les complexités des deux précédents décodeurs. On pourra se concentrer sur des codes RS définis sur $\mathbb{F}_{256}$. Testez les limites du nombre d'erreurs corrigibles.

Exercice 2 : Sage Math encore

1. Écrire ou rappeler une fonction `random_vector(n, q)` qui retourne un vecteur tiré uniformément dans $(\mathbb{F}_q)^n$.
2. Écrire ou rappeler une fonction `weight(x)` qui calcule le poids de Hamming d'un mot à coefficients q -aires.
3. Écrire ou rappeler une fonction `random_fixed_weight(n, w, q)` qui retourne un vecteur aléatoire de $(\mathbb{F}_q)^n$ de poids w .
4. Écrire ou rappeler une fonction `random_binary_matrix(k, n, q)` qui retourne une matrice aléatoire de coefficients dans $(\mathbb{F}_q)$ à k lignes et n colonnes, de rang maximal.
5. Écrire ou rappeler une fonction `random_subset(n, k)` qui retourne un sous-ensemble aléatoire $I \subset \llbracket 1, n \rrbracket$ de cardinal k .
6. Écrire ou rappeler une fonction `is_information_set(H, I)` qui teste si un ensemble I est un ensemble d'information d'un code de matrice de parité H .
7. Implémenter l'algorithme de Prange `prange(H, w, s)` en intégrant un compteur permettant d'estimer son nombre d'itérations et le temps de calcul.
8. Tester l'algorithme à l'aide des fonctions précédentes `test_prange()`
9. Tester plusieurs dizaines de fois l'algorithme sur des matrices aléatoires à l'aide des fonctions définies plus haut pour les paramètres suivants. On calculera à chaque fois les médianes du nombre d'itérations et du temps de calcul.
 - (a) $q = 2, k = \frac{n}{2}, w = \lfloor 0, 11n \rfloor$ et n croissant.
 - (b) $q = 2, k = \frac{n}{2}, w$ variant entre 1 et $\frac{n}{4}$ et n fixé.
10. Comparer enfin le temps mis par les algorithmes de Prange et Berlekamp pour le décodage syndrome dans le cas des codes de Reed-Solomon.

Exercice 3 : Algorithme de Sudan

Pour améliorer la capacité de correction des codes qui ne sont pas parfaits, on peut chercher à décoder $u = x + e$ avec e de poids supérieur à la capacité théorique de correction t . Les algorithmes de décodage en liste (par opposition aux algorithmes de décodage unique) renvoient une liste de mots à distance bornée de u . On peut ensuite choisir dans cette liste le mot le plus adapté (par exemple, le plus proche de u).

Soit $P \in \mathbb{F}_q[X, Y]$. $\deg_X(P)$ est le degré de P vu comme un polynôme en X , $\deg_Y(P)$ est le degré de P vu comme un polynôme en Y et $\deg(P)$ est le degré du monôme de plus haut degré, sachant que le degré d'un monôme est la somme de ses degrés en X et Y .

On considère un code $RS_k(\mathbf{x})$, avec $1 \leq k < n$ et on pose $t = \lfloor \frac{n-k}{2} \rfloor$.

1. Montrer que le décodage (unique) de Berlekamp-Welch pour décoder $RS_k(\mathbf{x})$ peut être reformulé ainsi :

Entrées : $u \in (\mathbb{F}_q)^n$;

Sorties : $f \in \mathbb{F}_q[x]_{<k}$ tel que $d(u, c) \leq t$ avec $c = (f(x_i))_i$;

- 1 **Interpolation** Construire à l'aide d'un système linéaire un polynôme $Q \in \mathbb{F}_q[X, Y]$ de la forme

$$Q = Q_0(X) + Q_1(X)Y \text{ tel que } \begin{cases} \deg(Q_0) < n - t \\ \deg(Q_1) \leq n - k - t \\ \forall i \in \llbracket 1, n \rrbracket, Q(x_i, u_i) = 0 \end{cases} ;$$

- 2 **Calcul des racines** Calculer les racines $f \in \mathbb{F}_q[X]$ de Q vu comme un polynôme en Y , c'est-à-dire calculer $f = -\frac{Q_0}{Q_1}$;

Sudan modifie l'algorithme pour lui permettre de renvoyer plus de solutions. On définit un paramètre $\ell > 1$ qui sera égal à $\deg_Y(Q)$

L'algorithme devient :

Entrées : $u \in (\mathbb{F}_q)^n$;

Paramètres : $\ell > 1$ et $r \geq t$

Sorties : tous les $f \in \mathbb{F}_q[x]_{<k}$ tels que $d(u, c) \leq r$ avec $c = (f(x_i))_i$;

- 1 **Interpolation** Construire à l'aide d'un système linéaire un polynôme $Q \in \mathbb{F}_q[X, Y]$ de la forme

$$Q = \sum_{j=0}^{\ell} Q_j(X)Y^j \text{ tel que}$$

$$\begin{cases} \forall j \in \llbracket 1, \ell \rrbracket, \deg(Q_j) + j(k-1) < n - r \\ \forall i \in \llbracket 1, n \rrbracket, Q(x_i, u_i) = 0 \end{cases}$$

;

- 2 **Calcul des racines** Calculer toutes les racines $f \in \mathbb{F}_q[X]$ de Q vu comme un polynôme en Y , c'est-à-dire calculer tous les f tels que $Q(X, f(X)) = 0$;

2. Montrer que toutes les solutions du problème de décodage sont bien retournées par l'algorithme.
3. Quel est le nombre maximum de solutions que peut retourner l'algorithme ? Respectent-elles toutes la condition $d(u, c) \leq r$?
4. Quelle est la valeur maximale ℓ_{max} du paramètre ℓ ? En choisissant $\ell = \ell_{max}$, la liste de mots de code retournée par l'algorithme de Sudan est-elle de taille polynomiale en n ?
5. Quel est le nombre d'équations du système linéaire de la phase d'interpolation ? Le nombre d'inconnues ? En déduire un critère qui garantisse que le système ait une solution non triviale.
6. Estimer la complexité de l'algorithme de Sudan.
7. Comparer le nombre d'erreurs corrigibles par l'algorithme de Sudan avec celui donné par Berlekamp-Welch.